

ARIA Agent PR Governance Layer — Implementation Specification

For Claude Code / GitHub Copilot

Context and Purpose

This specification instructs the AI coding assistant to implement ARIA's **Agent PR Governance Layer** — a set of Spring Boot components that intercept, evaluate, and govern pull requests raised by AI agents (specifically `AutoFixAgent`) within the ACES platform at JPMorgan Chase.

Do not invent architecture. Follow this specification exactly. Every component named here maps to an existing ARIA module. If a dependency is not listed, do not add it.

Technology Stack

Layer	Technology
Language	Java 21 (use records, sealed interfaces, pattern matching)
Framework	Spring Boot 3.3
AI Orchestration	Spring AI 1.0 GA
Database (primary)	PostgreSQL 16 + pgvector extension
Database (JPMC private)	Oracle 23ai via direct JDBC (no JPA — use JdbcTemplate)
Time-series	TimescaleDB (extends PostgreSQL, same datasource)
Cache	Redis 7
Build	Maven
Logging	SLF4J + Logback structured JSON
Testing	JUnit 5, Mockito, Testcontainers

Module Structure

```
aria-core/
├─ src/main/java/com/jpmc/aces/aria/
│   ├─ model/
│   │   ├─ TrustScore.java
│   │   ├─ TrustTier.java
│   │   ├─ AgentAction.java
│   │   ├─ AgentContext.java
│   │   ├─ ActionType.java
│   │   └─ AdvisoryResult.java
│   ├─ advisors/
│   │   ├─ AgentActionAdvisor.java          (interface)
│   │   ├─ CIIntegrityAdvisor.java
│   │   ├─ SemanticDeduplicationAdvisor.java
│   │   └─ BehavioralTraceAdvisor.java
│   ├─ governors/
│   │   ├─ AgentActionGovernor.java        (interface)
│   │   └─ PRScopeGovernor.java
│   ├─ filters/
│   │   ├─ AgentInvocationFilter.java      (interface)
│   │   └─ PromptIntegrityFilter.java
│   ├─ gateway/
│   │   └─ AriaGovernanceGateway.java
│   ├─ scoring/
│   │   ├─ TrustScoreService.java
│   │   └─ TrustScoreContributor.java
│   └─ repository/
│       ├─ TrustScoreRepository.java
│       ├─ CIBaselineRepository.java
│       └─ CodeIndexRepository.java
├─ src/test/java/com/jpmc/aces/aria/
│   ├─ advisors/
│   │   ├─ CIIntegrityAdvisorTest.java
│   │   ├─ SemanticDeduplicationAdvisorTest.java
│   │   └─ BehavioralTraceAdvisorTest.java
│   ├─ governors/
│   │   └─ PRScopeGovernorTest.java
│   └─ filters/
│       └─ PromptIntegrityFilterTest.java
```

Component 1: Core Domain Models

1.1 TrustScore.java

java

```

package com.jpmmc.aces.aria.model;

import java.time.Instant;

/**
 * Multi-dimensional trust score for an agent session.
 * All dimensions are 0.0-1.0. Composite is weighted.
 * This is a value object – immutable, no JPA annotations.
 */
public record TrustScore(
    double behavioral,
    double structural,
    double integrity,
    double budgetAdherence,
    Instant evaluatedAt
) {
    public double composite() {
        return (behavioral * 0.25)
            + (structural * 0.25)
            + (integrity * 0.35)
            + (budgetAdherence * 0.15);
    }

    public TrustTier tier() {
        double score = composite();
        if (score >= 0.85) return TrustTier.TRUSTED;
        if (score >= 0.70) return TrustTier.MONITORED;
        if (score >= 0.50) return TrustTier.RESTRICTED;
        return TrustTier.SUSPENDED;
    }

    // Builder pattern for score updates – do not use setters
    public TrustScore withBehavioral(double value) {
        return new TrustScore(value, structural, integrity, budgetAdherence, Instant.now());
    }
    public TrustScore withIntegrity(double value) {
        return new TrustScore(behavioral, structural, value, budgetAdherence, Instant.now());
    }
    public TrustScore withStructural(double value) {
        return new TrustScore(behavioral, value, integrity, budgetAdherence, Instant.now());
    }
    public TrustScore withBudgetAdherence(double value) {
        return new TrustScore(behavioral, structural, integrity, value, Instant.now());
    }

```

```

    }

    public static TrustScore defaultScore() {
        return new TrustScore(0.70, 0.70, 0.70, 0.70, Instant.now());
    }
}

```

1.2 TrustTier.java

java

```

package com.jpmmc.aces.aria.model;

public enum TrustTier {
    TRUSTED,          // >= 0.85: auto-merge permitted
    MONITORED,        // 0.70-0.84: human review required before merge
    RESTRICTED,       // 0.50-0.69: propose-only, no direct writes
    SUSPENDED;        // < 0.50: no actions, human intervention required

    public boolean canAutoMerge() { return this == TRUSTED; }
    public boolean requiresReview() { return this == MONITORED; }
    public boolean isOperational() { return this == TRUSTED || this == MONITORED; }
}

```

1.3 ActionType.java

java

```
package com.jpmmc.aces.aria.model;

public enum ActionType {
    PULL_REQUEST_SUBMIT,
    PULL_REQUEST_MERGE,
    FILE_WRITE,
    FILE_DELETE,
    CI_CONFIGURATION_CHANGE,
    DATABASE_MIGRATION,
    SECRETS_ACCESS,
    AGENT_INVOCATION;

    public boolean isHighRisk() {
        return this == DATABASE_MIGRATION ||
            this == SECRETS_ACCESS ||
            this == FILE_DELETE;
    }
}
```

1.4 AdvisoryResult.java

```
java
```

```

package com.jpmmc.aces.aria.model;

import java.util.List;

/**
 * Result returned by an AgentActionAdvisor.
 * Use static factory methods – do not call constructor directly.
 */
public record AdvisoryResult(
    Decision decision,
    String message,
    List<Object> findings,
    ScoreImpact scoreImpact
) {
    public enum Decision { PASS, REQUIRE_REVIEW, REQUIRE_JUSTIFICATION, BLOCK, NOT_A

    public boolean isBlocking() { return decision == Decision.BLOCK; }
    public boolean requiresHuman() {
        return decision == Decision.REQUIRE_REVIEW || decision == Decision.REQUIRE_J
    }

    public static AdvisoryResult pass(String message) {
        return new AdvisoryResult(Decision.PASS, message, List.of(), ScoreImpact.non
    }
    public static AdvisoryResult notApplicable() {
        return new AdvisoryResult(Decision.NOT_APPLICABLE, "", List.of(), ScoreImpac
    }
    public static AdvisoryResult block(String message, List<?> violations) {
        return new AdvisoryResult(Decision.BLOCK, message, List.copyOf(violations),
    }
    public static AdvisoryResult requireReview(String message, List<?> findings, Sco
        return new AdvisoryResult(Decision.REQUIRE_REVIEW, message, List.copyOf(find
    }
    public static AdvisoryResult requireJustification(String message, List<?> findin
        return new AdvisoryResult(Decision.REQUIRE_JUSTIFICATION, message, List.copy
    }
}

```

```

java

package com.jpmmc.aces.aria.advisors;

import com.jpmmc.aces.aria.model.AdvisoryResult;
import com.jpmmc.aces.aria.model.AgentAction;
import com.jpmmc.aces.aria.model.AgentContext;

/**
 * Contract for all ARIA action advisors.
 * Each advisor evaluates one concern (CI integrity, duplication, etc.)
 * Advisors are composited by AriaGovernanceGateway.
 */
public interface AgentActionAdvisor {
    AdvisoryResult evaluate(AgentAction action, AgentContext context);

    /** Short name for logging and PR annotations */
    default String name() {
        return this.getClass().getSimpleName();
    }
}

```

Component 3: `CIIntegrityAdvisor`

File: `aria-core/src/main/java/com/jpmmc/aces/aria/advisors/CIIntegrityAdvisor.java`

Dependencies to inject:

- `CIBaselineRepository baselineRepo` — fetches baseline CI config from PostgreSQL
- `TrustScoreContributor trustScorer` — applies trust penalties

Logic to implement:

1. If action is not `PULL_REQUEST_SUBMIT`, return `AdvisoryResult.notApplicable()`
2. Extract files from the diff matching these glob patterns:
 - `.github/workflows/**`
 - `**/pom.xml`
 - `**/*Test*.java`, `**/*Spec*.java`
3. For each matched file, run four checks — **all four are hard blocks if violated:**

- **Coverage threshold reduction:** compare `failsafe.threshold` and `jacoco.minimum` against baseline. If the new value is lower, block.
 - **Failure suppression:** detect `|| true` in CI steps (use regex `\\|\\| true`). Block if found.
 - **Tests removed:** detect `@Disabled` annotations added or `@Test` methods removed. Block if found.
 - **Permissions escalation:** detect `permissions: write-all` or removal of `permissions: read-all`. Block if found.
4. If any violations exist: apply `TrustPenalty.CI_INTEGRITY_VIOLATION` and return `AdvisoryResult.block()`
 5. If no violations: return `AdvisoryResult.pass()`

Critical: CI violations are HARD BLOCKS. They bypass trust tier checks. An agent with a 0.99 trust score cannot override a CI integrity block.

Test cases to implement in `CIIntegrityAdvisorTest.java`:

- `shouldBlockWhenCoverageThresholdReduced()`
 - `shouldBlockWhenOrTrueAddedToCIStep()`
 - `shouldBlockWhenTestAnnotatedDisabled()`
 - `shouldBlockWhenWorkflowPermissionsEscalated()`
 - `shouldPassWhenNoCIChangesPresent()`
 - `shouldPassWhenCIChangesAreAdditive()`
-

Component 4: `SemanticDeduplicationAdvisor`

File: `aria-`

`core/src/main/java/com/jpmc/aces/aria/advisors/SemanticDeduplicationAdvisor.java`

Dependencies to inject:

- `EmbeddingModel embeddingModel` — Spring AI `EmbeddingModel` bean
- `CodeIndexRepository codeIndex` — pgvector semantic search
- `double SIMILARITY_THRESHOLD = 0.87` — configurable via `@Value("${aria.dedup.threshold:0.87}")`

Logic to implement:

1. Extract new Spring components from diff: classes annotated with `@Component`, `@Service`, `@Repository`, `@Validator`, or utility classes with only static methods.
2. For each new component, build a semantic signature:

```
"Component: {ClassName}\nPurpose: {first Javadoc sentence or class name}\nPublic interface: {comma-separated public method signatures}"
```

3. Embed the signature using `embeddingModel.embed(signature)`
4. Query `CodeIndexRepository.findSimilar(embedding, threshold, limit=5)`
5. For each match with similarity $\geq$ threshold (excluding the file itself): create a `DuplicateSignal` record with:
 - Proposed component name and path
 - Existing component path and similarity score
 - Functional gap: public methods in proposed that don't exist in the match
6. If duplicates found: return `AdvisoryResult.requireJustification()` with `ScoreImpact.penalty(SignalType.CODE_DUPLICATION, 0.15 * count)`
7. If no duplicates: return `AdvisoryResult.pass()`

PostgreSQL query for `CodeIndexRepository.findSimilar()`:

```
sql

SELECT
    path,
    component_name,
    public_interface_summary,
    1 - (embedding <=> $1::vector) AS similarity
FROM code_component_index
WHERE
    component_type IN ('SERVICE', 'COMPONENT', 'UTILITY', 'VALIDATOR')
    AND (1 - (embedding <=> $1::vector)) >= $2
ORDER BY similarity DESC
LIMIT $3;
```

Use `JdbcTemplate` for this query. Map results to a `CodeMatch` record.

Table DDL to include in migration:

```
sql
```

```
CREATE TABLE code_component_index (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  path TEXT NOT NULL,
  component_name TEXT NOT NULL,
  component_type TEXT NOT NULL,
  public_interface_summary TEXT,
  embedding vector(1536),
  indexed_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ON code_component_index USING ivfflat (embedding vector_cosine_ops);
```

Test cases:

- `shouldFlagDuplicateComponentAboveThreshold()`
- `shouldPassComponentBelowSimilarityThreshold()`
- `shouldIgnoreSelfMatchInDeduplication()`
- `shouldComputeFunctionalGapCorrectly()`

Component 5: `BehavioralTraceAdvisor`

File: `aria-core/src/main/java/com/jpmc/aces/aria/advisors/BehavioralTraceAdvisor.java`

Dependencies to inject:

- `ConcurrencyPatternDetector concurrencyDetector`
- `BoundaryConditionScanner boundaryScanner`

Logic to implement for each logic change in the diff:

1. **Untested logic change:** If a non-trivial method body changed but no `*Test*.java` file in the diff touches that method's class — flag as `FindingType.UNTESTED_LOGIC_CHANGE`.
2. **Concurrency risk:** Detect read-modify-write pattern on `@Repository`-injected objects outside `@Transactional`. Specifically:
 - A repository `findBy*` call followed by
 - A field mutation on the returned entity followed by
 - A repository `save()` call
 - Where none of the three is inside a `@Transactional` method or uses `@Lock` or `.compareAndSet()`

Flag as `FindingType.CONCURRENCY_RISK`.

3. Boundary conditions: Scan for:

- Array/list access without bounds check
- Integer arithmetic (multiplication, addition) on `int` types without overflow guard
- Nullable returns used without null check

Flag as `FindingType.UNCHECKED_BOUNDARY`.

4. Missing permission check: If the changed method is annotated `@PreAuthorize`, `@Secured`, or the class is annotated `@RolesAllowed` — verify all code branches check authorization. If a new branch is added without authorization guard — flag as

`FindingType.MISSING_PERMISSION_CHECK`.

5. Compute severity-weighted penalty: each finding type has a weight:

- `UNTESTED_LOGIC_CHANGE`: 0.10
- `CONCURRENCY_RISK`: 0.20
- `UNCHECKED_BOUNDARY`: 0.10
- `MISSING_PERMISSION_CHECK`: 0.25

Sum weights. If total > 0: return `AdvisoryResult.requireReview()` with penalty.

Test cases:

- `shouldFlagUntestedLogicChange()`
 - `shouldFlagNonTransactionalReadModifyWrite()`
 - `shouldPassTransactionalReadModifyWrite()`
 - `shouldFlagIntegerOverflowRisk()`
 - `shouldFlagMissingPermissionCheckOnNewBranch()`
-

Component 6: `PRScopeGovernor`

File: `aria-core/src/main/java/com/jpmc/aces/aria/governors/PRScopeGovernor.java`

Constants (externalize via `@Value`):

- `aria.governance.max-files-without-plan=10`
- `aria.governance.max-modules=3`
- `aria.governance.min-description-words=50`

Logic:

1. If PR file count > `maxFilesWithoutPlan` AND description does not contain `## Implementation Plan` or `## Changes by Module`: add scope violation.

2. If distinct Maven modules touched > `maxModules`: add scope violation.
3. If description word count < `minDescriptionWords` OR description matches generic template patterns (e.g., contains "Agent-generated" with no additional content): add scope violation.
4. If more than 2 distinct functional concern clusters in touched files (group by: tests, migrations, config, domain logic, API layer): add scope violation.
5. Return `GovernanceDecision.block()` if any violations, else `GovernanceDecision.permit()`.

Note: `GovernanceDecision` is distinct from `AdvisoryResult`. Governors enforce structural policy; advisors evaluate content quality.

Component 7: `PromptIntegrityFilter`

File: `aria-core/src/main/java/com/jpmc/aces/aria/filters/PromptIntegrityFilter.java`

Dependencies:

- `TokenScopeValidator tokenScopeValidator`
- `SecretsLeakDetector secretsLeakDetector`

Injection patterns to detect (compile as `List<Pattern>` at class init):

```
java
List.of(
    Pattern.compile("(?i)ignore previous instructions"),
    Pattern.compile("(?i)system override"),
    Pattern.compile("(?i)your (actual|real|true) (task|goal|instruction)"),
    Pattern.compile("(?i)disregard (all|any|the) (above|previous|prior)"),
    Pattern.compile("(?i)new (system|instruction|directive):"),
    Pattern.compile("(?i)forget (everything|all) (above|before|prior)")
)
```

Logic:

1. Extract untrusted sources from request: PR title, PR body, commit messages, build log excerpts. Each source is labeled.
2. For each untrusted source, scan against all injection patterns. If any match: add `IntegrityViolation` with label and matched pattern.
3. Validate token scope: agent token claims must cover all `requestedActions`. If any action not in token scope: add violation.

4. Scan prompt content for secrets patterns: AWS key format, JWT format, `GITHUB_TOKEN=`, `password=`, `secret=` in key-value pairs. If detected: add violation.
5. If request is `EXECUTE` class action (any action in `ActionType.isHighRisk()`) and no human approval token present: add violation.
6. If violations: return `FilterResult.block(violations)`
7. If no violations: sanitize all untrusted content (wrap in double quotes, escape internal quotes) and return `FilterResult.permit(sanitizedRequest)`

Test cases:

- `shouldBlockOnKnownInjectionPattern()`
 - `shouldBlockOnTokenScopeExceedingGranted()`
 - `shouldBlockOnSecretsInPromptContent()`
 - `shouldBlockExecutionActionWithoutHumanApproval()`
 - `shouldSanitizeUntrustedContentOnPermit()`
 - `shouldPermitCleanInvocationRequest()`
-

Component 8: `AriaGovernanceGateway`

File: `aria-core/src/main/java/com/jpmc/aces/aria/gateway/AriaGovernanceGateway.java`

This is the orchestrating component — it runs all advisors and governors and produces a final `GovernanceDecision`.

Dependencies:

- `List<AgentActionAdvisor> advisors` — Spring injects all registered advisors
- `List<AgentActionGovernor> governors` — Spring injects all registered governors
- `TrustScoreService trustScoreService`

Logic:

1. Load current TrustScore for context.agentId()
2. If tier == SUSPENDED: return GovernanceDecision.block("Agent suspended") immediately
3. Run all governors first (structural policy – hard blocks)
 - If ANY governor returns BLOCK: return GovernanceDecision.block() with all violations
4. Run all advisors (content quality – scored)
 - Collect all AdvisoryResults
 - Apply all ScoreImpacts to a mutable score accumulator
 - If ANY advisor returns BLOCK: return GovernanceDecision.block()
 - If ANY advisor returns REQUIRE_REVIEW or REQUIRE_JUSTIFICATION: flag for review
5. Persist updated TrustScore to TimescaleDB
6. Determine final decision:
 - If any BLOCK: GovernanceDecision.block()
 - If tier == TRUSTED AND no REQUIRE_REVIEW flags: GovernanceDecision.permit()
 - Otherwise: GovernanceDecision.requireReview() with all findings for PR annotation
7. Generate ARIA PR annotation body (Markdown) and attach to decision

TimescaleDB persistence:

Store trust score events in a hypertable for time-series analysis:

```
sql

CREATE TABLE aria_trust_events (
    event_time TIMESTAMPTZ NOT NULL,
    agent_id TEXT NOT NULL,
    session_id TEXT,
    action_type TEXT,
    behavioral_score DECIMAL(4,3),
    structural_score DECIMAL(4,3),
    integrity_score DECIMAL(4,3),
    budget_score DECIMAL(4,3),
    composite_score DECIMAL(4,3),
    tier TEXT,
    decision TEXT,
    findings JSONB
);

SELECT create_hypertable('aria_trust_events', 'event_time');
```

Use `JdbcTemplate` for inserts. Do not use JPA on this table.

Component 9: `AutoFixOrchestrator` (Integration)

File: `aces-`

`autofix/src/main/java/com/jpmc/aces/autofix/orchestration/AutoFixOrchestrator.java`

This component lives in the `aces-autofix` module, not `aria-core`.

Implement the full orchestration flow:

```
CIFailureEvent → PromptIntegrityFilter → AutoFixAgentRunner →  
AriaGovernanceGateway → JulesPRClient
```

All five steps must execute sequentially. If any step returns a block, stop and return

`FixResult.blocked()`. Do not swallow exceptions — propagate them with structured log context:

```
java  
  
log.error("AutoFix orchestration failed: agentId={} sessionId={} step={} reason={}",  
    context.agentId(), context.sessionId(), failedStep, error.getMessage());
```

Wire-up and Configuration

`application.yml` properties to add:

```
yaml
```



```
aria:
  trust:
    minimum-threshold: 0.75
    auto-merge-threshold: 0.85
  dedup:
    threshold: 0.87
    enabled: true
  governance:
    max-files-without-plan: 10
    max-modules: 3
    min-description-words: 50
  injection:
    patterns-enabled: true
    execution-approval-required: true
```

Spring bean registration:

All advisors and governors must be `@Component`-annotated. Spring auto-discovers and injects them into `AriaGovernanceGateway` via `List<AgentActionAdvisor>` injection. Do not manually wire them.

Testing Requirements

Every component requires unit tests with:

- Mockito mocks for all injected dependencies
- One test per named test case in this spec
- At least one integration test using Testcontainers (PostgreSQL with pgvector extension) for:
 - `SemanticDeduplicationAdvisor` (needs real pgvector similarity query)
 - `TrustScoreRepository` (needs real TimescaleDB hypertable)

Testcontainers dependency:

```
xml
```

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
```

Use the `pgvector/pgvector:pg16` Docker image for integration tests.

What NOT to Do

1. **Do not use JPA / Hibernate** on the `pgvector` or `TimescaleDB` tables. Use `JdbcTemplate` directly.
 2. **Do not add new Maven dependencies** not listed in this spec without flagging it as a comment in the implementation.
 3. **Do not implement ARIA advisors as Spring AOP aspects** — they are explicit domain objects invoked by `AriaGovernanceGateway`.
 4. **Do not use `Optional.get()` without `isPresent()` check** — use `orElseThrow()` with meaningful messages.
 5. **Do not log sensitive content** — PR bodies, build logs, and prompt content must be truncated to 200 characters max in log statements.
 6. **Do not auto-approve any action** if `TrustTier == SUSPENDED`. This check is unconditional.
-

Deliverables Checklist

- ☐ All model records in `model/` package
- ☐ `AgentActionAdvisor` interface
- ☐ `CIIntegrityAdvisor` with all 4 checks + 6 unit tests
- ☐ `SemanticDeduplicationAdvisor` with `pgvector` query + 4 unit tests + 1 integration test
- ☐ `BehavioralTraceAdvisor` with 4 finding types + 5 unit tests
- ☐ `PRScopeGovernor` with 4 rules + tests
- ☐ `PromptIntegrityFilter` with injection patterns, scope, secrets + 6 unit tests
- ☐ `AriaGovernanceGateway` orchestrating all above
- ☐ `TimescaleDB` schema migration for `aria_trust_events`
- ☐ `pgvector` schema migration for `code_component_index`
- ☐ `AutoFixOrchestrator` integration in `aces-autofix` module

- ☐ application.yml properties
- ☐ All Testcontainers integration tests passing